

Tecnologías utilizadas

1. Resumen

El proyecto Contacto 3xtrat3rr3str3D es una aplicación de escritorio desarrollada íntegramente en Java, que integra tres lenguajes de programación propios (.y, .z, .pig), valida su semántica y traduce el código a C, compilándolo con gcc.

La arquitectura se apoya en tres pilares técnicos:

1. ANTLR 4 para el análisis léxico, sintáctico y construcción del AST de los tres lenguajes.
2. JavaFX para la interfaz gráfica (editores, árbol de trabajo, panel de salida, errores).
3. GCC (GNU Compiler Collection) invocado desde Java mediante ProcessBuilder para generar el ejecutable final.

2. Detalle por tecnología

2.1 Java

Aspecto	Detalle
Versión	Java 21 (según classpath del proyecto: jdk-21.0.12)
Uso	Lenguaje principal de todo el proyecto
Características usadas	Records, sealed interfaces, pattern matching para switch, switch expressions, instanceof con binding, streams
Gestión de dependencias	Maven

Java 21 se eligió por el uso intensivo de sealed interfaces y records para modelar el AST de los tres lenguajes. Cada nodo del AST es un record inmutable, y cada expresión o sentencia se modela como una variante de una interfaz sellada. Esto permite usar pattern matching exhaustivo en los switch.

Ejemplo del uso de sealed interfaces en el AST de .y:

```
java
```

```
public sealed interface NodoExpr extends NodoAST permits
    NodoExpr.LiteralEntero,
    NodoExpr.LiteralFlotante,
    NodoExpr.LiteralCadena,
    ...
```

2.2 ANTLR 4

Aspecto	Detalle
Versión	4.13.1
Uso	Generación del lexer y parser para los tres lenguajes
Archivos fuente	YLexer.g4, YParser.g4, ZLexer.g4, ZParser.g4, PigLexer.g4, PigParser.g4
Modo de uso	Visitor (YParserBaseVisitor, ZParserBaseVisitor, PigParserBaseVisitor)

Por qué ANTLR 4: es un requisito explícito del proyecto ("Realizar cualquier análisis sintáctico y semántico con ANTLR4", según la documentación). ANTLR 4 permite definir la gramática de un lenguaje en notación EBNF y genera automáticamente las clases de lexer y parser.

Flujo con ANTLR:

text

Archivo .y/.z/.pig



Lexer (generado por ANTLR) → tokens



Parser (generado por ANTLR) → ParseTree



Visitor (implementación propia) → AST propio

Detalle importante en .y: se usa un preprocesador de indentación

(PreprocesadorIndentacion) que convierte las tabulaciones significativas del lenguaje .y (estilo Python) en tokens INDENT y DEDENT antes de pasar el código al lexer.

2.3 JavaFX

Aspecto	Detalle
Versión	21.0.6
Uso	Interfaz gráfica completa
Módulos usados	javafx-controls, javafx-graphics, javafx-base, javafx-fxml, javafx-web, javafx-swing

JavaFX se utiliza para construir toda la UI del proyecto. Los componentes principales son:

Componente	Tecnología JavaFX
Ventana principal	BorderPane
Menú principal	MenuBar, Menu, MenuItem, Button
Árbol de archivos	TreeView, TreeItem
Panel de editores con pestañas	TabPane, Tab
Editor de código	CodeArea (de RichTextFX, ver 2.4)
Panel de salida con pestañas	TabPane con TextArea y TableView
Split de editores + salida	SplitPane
Diálogos	FileChooser, ChoiceDialog

2.4 RichTextFX (CodeArea)

Aspecto	Detalle
Librería	org.fxmisc.richtext (richtextfx 0.11.7)
Uso	Editor de código con números de línea, resaltado y undo/redo nativo

CodeArea es el componente base del editor. Provee:

- Números de línea sincronizados con los párrafos.
- Resaltado de sintaxis mediante StyleSpans (sin librerías externas, propio).
- Undo/Redo nativo (undo(), redo()).
- Cut/Copy/Paste nativos.
- Multi-caret y selección múltiple.

2.5 GCC (GNU Compiler Collection)

Aspecto	Detalle
Uso	Compilación del código C generado a ejecutable

Invocación	ProcessBuilder desde Java
Comando	gcc programa.c -o programa

El compilador C se invoca desde Java usando ProcessBuilder, que permite:

1. Definir el directorio de trabajo.
2. Capturar la salida de gcc (stdout + stderr) línea por línea.
3. Esperar a que termine y verificar el código de salida.
4. Reportar errores de compilación al usuario si falla.

Fragmento representativo:

```
java
ProcessBuilder pb = new ProcessBuilder(
    "gcc", rutaC.getFileName().toString(), "-o", rutaExe.getFileName().toString()
);
pb.directory(rutaC.getParent().toFile());
pb.redirectErrorStream(true);
Process proceso = pb.start();
```

2.6 Otras librerías usadas

Librería	Versión	Uso
Lombok	(según Maven)	@Getter, @AllArgsConstructor en clases de datos
ControlsFX	11.2.1	Componentes adicionales JavaFX (opcional)
FormsFX	11.6.0	Formularios JavaFX (declarado, uso mínimo)
BootstrapFX	0.4.0	Estilos CSS tipo Bootstrap (declarado)
Ikonli	12.3.1	Iconos (declarado)
TilesFX	21.0.9	Componentes tipo dashboard (declarado)
ValidatorFX	0.6.1	Validación de campos (declarado)

Nota: varias de estas librerías están declaradas en pom.xml pero no se usan activamente en el código. Se mantienen por si se necesitan en la parte 2 del proyecto.

2.7 Maven

Aspecto	Detalle
Uso	Gestión de dependencias y construcción del proyecto
Archivo	pom.xml
Plugins principales	maven-compiler-plugin, javafx-maven-plugin (o equivalente), antlr4-maven-plugin

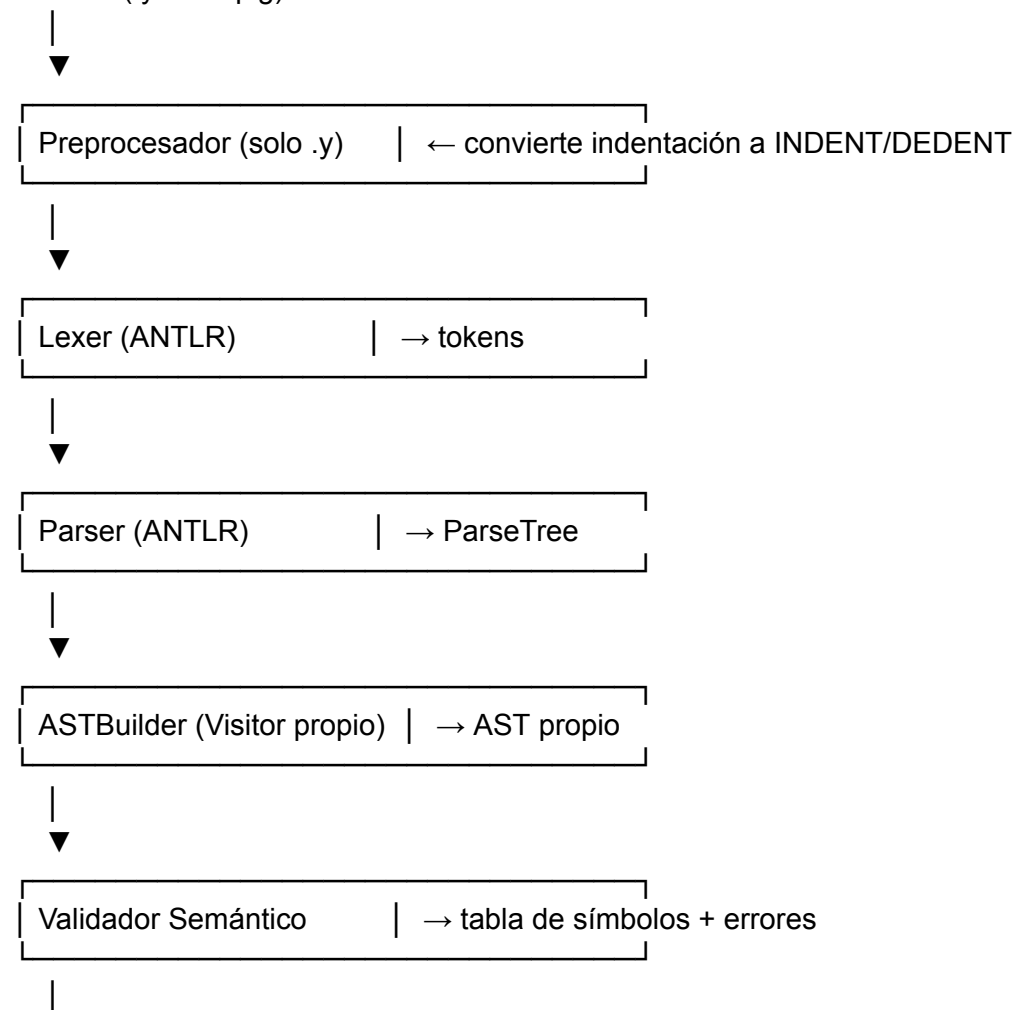
Nota: si generas el parser con ANTLR manualmente (con el JAR de ANTLR), no necesitas el antlr4-maven-plugin. Si lo generas con Maven, sí.

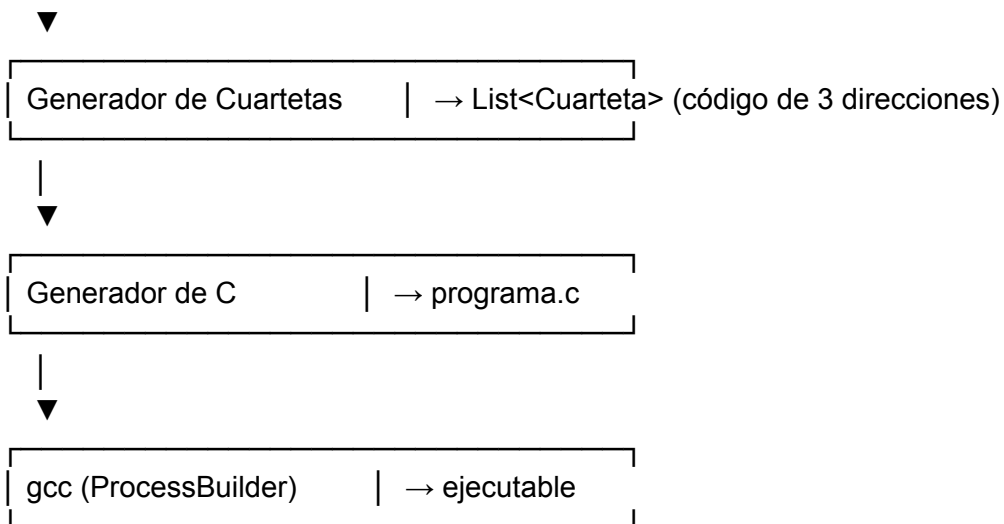
3. Arquitectura general del sistema

3.1 Pipeline de compilación

text

Entrada (.y / .z / .pig)

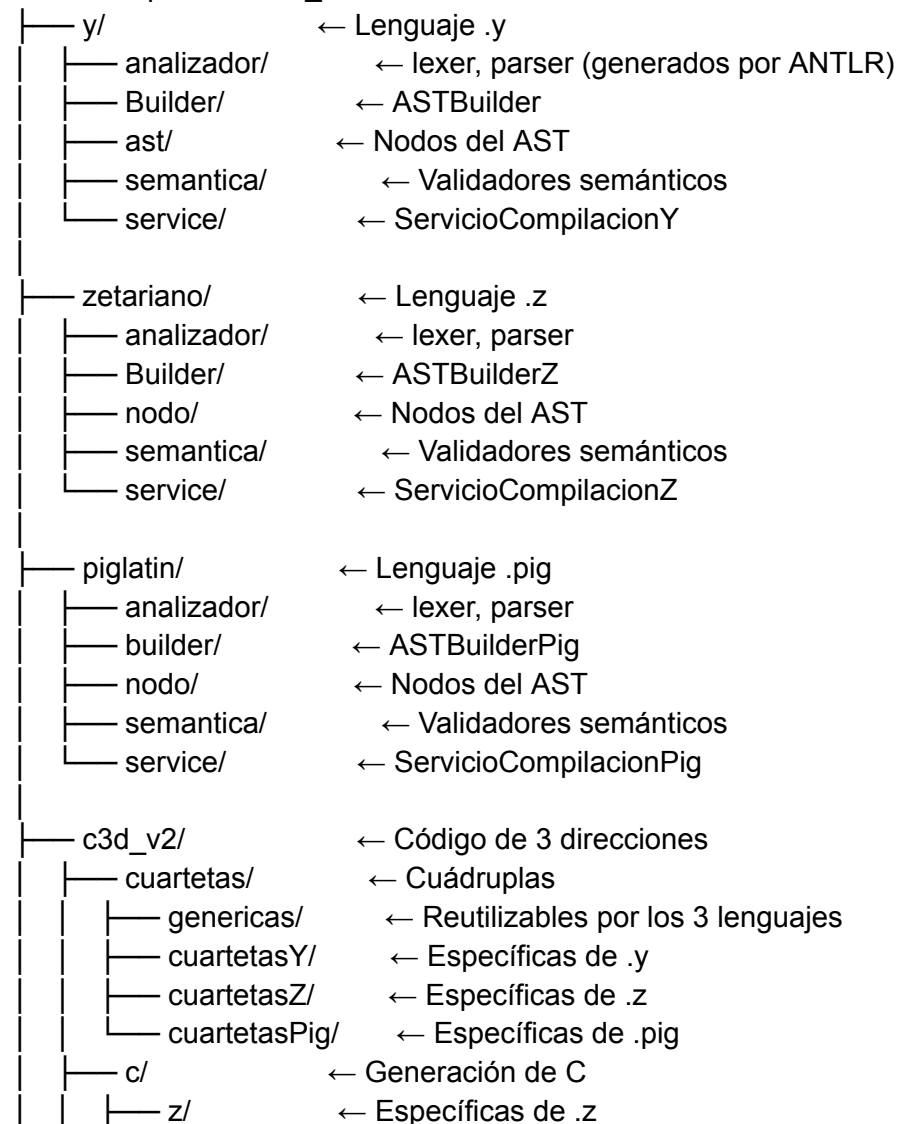


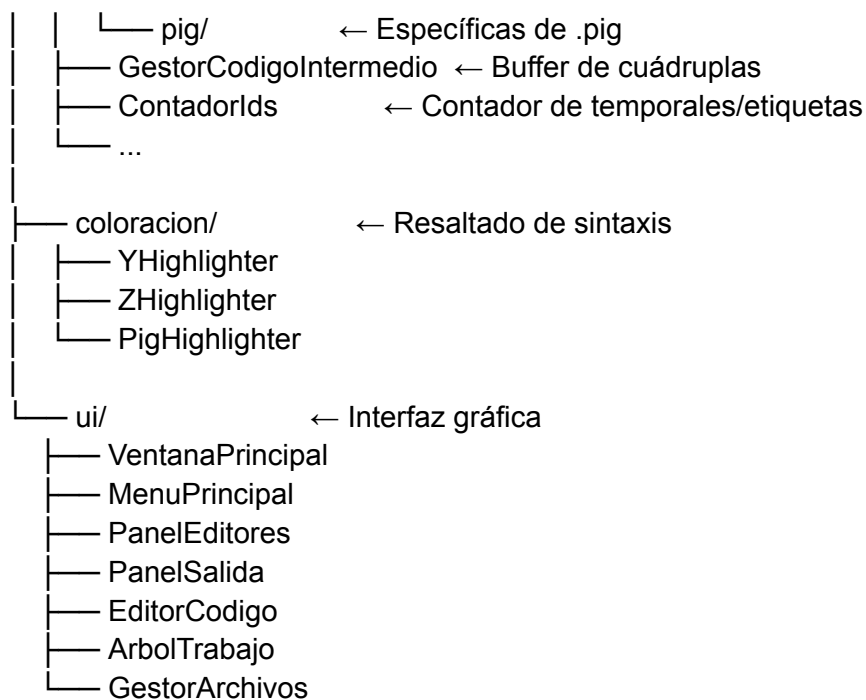


3.2 Estructura de paquetes

text

com.example.contacto_3xtrat3r3str3





3.3 Resaltado de sintaxis

El resaltado de sintaxis se implementa sin librerías externas, usando el motor de StyleSpans de RichTextFX. Cada lenguaje tiene su propia clase de resaltado:

Lenguaje	Clase	Método
.y	YHighlighter	computeHighlighting(String texto)
.z	ZHighlighter	computeHighlighting(String texto)
.pig	PigHighlighter	computeHighlighting(String texto)

El resaltado se recalcula cada 300 ms tras la última pulsación (gracias a `multiPlainChanges().successionEnds(...)` de RichTextFX). No se permite el uso de librerías para el coloreado, según la especificación del proyecto.

4. Requisitos del sistema

4.1 Para ejecutar la aplicación

Requisito	Versión mínima
Java (JRE)	21
JavaFX	21.0.6 (incluido en las dependencias)

Sistema operativo	Windows, Linux o macOS
GCC	Instalado y accesible en el PATH

4.2 Para compilar el proyecto desde código fuente

Requisito	Versión
JDK	21
Maven	3.8+
ANTLR	4.13.1 (para regenerar parser si se modifican gramáticas)
GCC	Cualquier versión reciente

5. Consideraciones de diseño

5.1 ¿Por qué Java?

Porque es un requisito explícito del proyecto ("Lenguaje de programación a utilizar: JAVA"). Además, Java + ANTLR + JavaFX es una combinación estándar en cursos de compiladores.

5.2 ¿Por qué ANTLR 4?

Porque es un requisito explícito ("Realizar cualquier análisis sintáctico y semántico con ANTLR4"). Y porque permite definir gramáticas EBNF de forma declarativa, generando lexer y parser robustos.

5.3 ¿Por qué JavaFX?

Porque es la biblioteca de UI moderna de Java, con soporte nativo para todo lo que necesita el proyecto (tablas, tabs, árboles, editores, etc.).

5.4 ¿Por qué RichTextFX?

Porque TextArea estándar de JavaFX no soporta resaltado de sintaxis. RichTextFX (CodeArea) sí, y además provee números de línea, undo/redo y multi-caret.

5.5 ¿Por qué GCC y no otro compilador de C?

Porque es el compilador estándar en entornos Unix y está disponible en Windows mediante MinGW. Además, la salida de gcc es fácil de capturar y mostrar al usuario.

5.6 ¿Por qué un AST propio y no usar directamente el ParseTree de ANTLR?

Porque el ParseTree de ANTLR contiene nodos auxiliares y tokens que no aportan información semántica. Un AST propio con records sellados es más limpio, tipado y fácil de recorrer. Esto facilita:

- La validación semántica (cada tipo de nodo sabe validarse).
- La traducción a cuádruplas (cada nodo sabe emitir cuádruplas).
- El resaltado de sintaxis (aunque se hace aparte, con ANTLR directamente).

6. Resumen final

Categoría	Tecnología
Lenguaje de programación	Java 21
Generador de parser	ANTLR 4.13.1
UI	JavaFX 21.0.6
Editor de código	RichTextFX (CodeArea)
Compilador de C	GCC (invocado con ProcessBuilder)
Build	Maven
Otras	Lombok, ControlsFX, FormsFX, Ikonli, TilesFX (declaradas)

¿Seguimos con la siguiente sección del manual técnico? Puedo generar:

- Especificaciones de palabras reservadas (ya tienes una versión previa, puedo pulirla).
- Tabla de compatibilidad de tipos (ya tienes una versión previa).
- Diagrama de clases.
- Especificación de la gramática (reglas ANTLR).
- Análisis léxico/sintáctico/semántico con manejo de errores.
- Cuádruplas y generación C.

Tabla de compatibilidad

Lenguaje .y

Destino ↓ \ Origen →	entero	flotante	caracter	cadena	bool
entero	✓	✗	✗	✗	✗
flotante	✓ (promoción)	✓	✗	✗	✗
caracter	✗	✗	✓	✗	✗
cadena	✓ (concatenación)	✓ (concatenación)	✓ (concatenación)	✓	✓ (concatenación)
bool	✗	✗	✗	✗	✓

Lenguaje .z

Destino ↓ \ Origen →	int	double	char	boolean	String	null
int	✓	✗	✗	✗	✗	✗
double	✓ (promoción)	✓	✗	✗	✗	✗
char	✗	✗	✓	✗	✗	✗

boolean	✗	✗	✗	✓	✗	✗
String	✓ (concat ena)	✓ (concat ena)	✓ (concat ena)	✓ (concat ena)	✓	✗
Clase (objeto)	✗	✗	✗	✗	✗	✓

Lenguaje .pig

Destino ↓ \ Origen →	numerus	decimalis	littera	textum	bool
numerus	✓	✗	✗	✗	✗
decimalis	✓ (promoción)	✓	✗	✗	✗
littera	✗	✗	✓	✗	✗
textum	✓ (concaten a)	✓ (concaten a)	✓ (concaten a)	✓	✓ (concatena)
bool	✗	✗	✗	✗	✓

Tabla de condicionales

Tabla de condicionales — .y

Estructura	Sintaxis	¿Soportado?	Notas
si simple	si (cond) entonces ...	✓	Cierra con DEDENT (indentación)
si + sino	si (cond) entonces ... sino (cond2) entonces ...	✓	La condición del sino es obligatoria
si + contrario	si (cond) entonces ... contrario ...	✓	contrario es el else final, sin condición
si + sino + contrario	si ... sino (cond) ... contrario ...	✓	Combinación completa
Anidados	si (cond) entonces si (cond2) entonces ...	✓	Por indentación

elegir (switch)	elegir (expr): caso lit: ... siempre: ...	✓	Con romper opcional
caso con literal	caso 1: ...	✓	Solo acepta literales (no expresiones)
caso con varios literales	caso 1, 2: ...	✗	No soportado
siempre (default)	siempre: ...	✓	Opcional
ternario (?:)	cond ? a : b	✗	No existe en .y

Tabla de condicionales — .z

Estructura	Sintaxis	¿Soportado?	Notas
if simple	if (cond) { ... }	✓	Llaves obligatorias
if + else	if (cond) { ... } else { ... }	✓	
if + else if + else	if (c1) {...} else if (c2) {...} else {...}	✓	
if sin llaves (1 línea)	if (cond) sentencia;	✓	La gramática permite sentenciaOBloque
Anidados	if (...) { if (...) {...} }	✓	
switch + case + default	switch (expr) { case 1: ... default: ... }	✓	
case con literal	case 1:	✓	

case con múltiples valores	case 1, 2:	✗	No soportado
default	default:	✓	Opcional
break en switch	break;	✓	Opcional (fall-through como C)
Ternario ?:	cond ? a : b	✓	Sí soporta ternario
Operadores lógicos	&& !	✓	

Tabla de condicionales — .pig

Estructura	Sintaxis	¿Soportado?	Notas
si simple	si (cond) { ... } finis;	✓	Cierra con finis;
si + aliter (cond)	si (c1) {...} aliter (c2) {...} finis;	✓	aliter con condición
si + aliter (else)	si (c1) {...} aliter {...} finis;	✓	aliter sin condición = else
Múltiples aliter	si (...) ... aliter (...) ... aliter (...) ... aliter ...	✓	Ilimitados
si + aliter + aliter (con cond)	si (c1) {...} aliter (c2) {...} aliter (c3) {...} aliter {...} finis;	✓	Combinación completa
Anidados	si (...) { si (...) {...} finis; } finis;	✓	
Ternario ?:	cond ? a : b	✗	No existe en .pig
switch / elegir	—	✗	No soportado en .pig
Operadores lógicos	&& !	✓	

